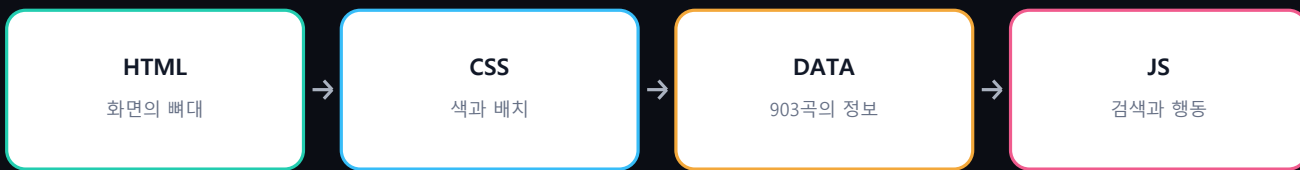


ACTUAL PROJECT LEARNING GUIDE

Flylist를 읽고 고치는 법

비전공자를 위한 HTML · CSS · JavaScript · 데이터 구조 입문서



네 파일이 연결되어 하나의 Flylist가 됩니다.

기준 버전 b1312c4 | 2026-07-30 | 903곡

제작 방향: 코드를 외우는 책이 아니라, 현재 Flylist를 안전하게 직접 수정하기 위한 책

목차

처음에는 1장부터 4장까지만 읽어도 충분합니다. 이후에는 하고 싶은 수정에 맞는 장을 골라 보세요.

0. 이 책을 사용하는 방법	5
세 가지 읽기 경로	5
코드를 읽는 기본 자세	5
1. 웹사이트를 보는 가장 쉬운 관점	6
Flylist는 정적 사이트입니다	6
네 가지 핵심 파일	6
2. 현재 Flylist의 전체 구조	7
브라우저가 페이지를 여는 순서	7
나머지 파일의 역할	7
화면을 만드는 핵심 흐름	7
3. HTML 문법: 화면의 뼈대 읽기	8
태그의 기본 모양	8
부모와 자식	8
Flylist에서 자주 보는 HTML 태그	9
hidden과 defer	9
4. CSS 문법: 색과 배치 바꾸기	10
선택자, 속성, 값	10
색상 변수는 조종석입니다	10
박스 모델	11
모바일 규칙: 미디어 쿼리	11
5. data.js 문법: 곡 데이터 읽기	12
배열과 객체	12
곡 필드 사전	12
심표가 중요한 이유	13
현재 데이터 한눈에 보기	13
6. JavaScript 문법: app.js 읽기 준비	14
const와 let: 이름 붙인 상자	14
배열과 객체	14

함수: 이름 붙인 작업 절차	14
조건문과 짧은 조건식	14
map, filter, forEach	15
Map과 Set	15
7. app.js의 큰 흐름	16
1단계: 데이터와 상태 준비	16
2단계: HTML 요소에 짧은 이름 붙이기	16
3단계: init으로 시작	16
4단계: 이벤트를 듣기	16
5단계: render 함수로 화면 만들기	17
8. 검색은 실제로 어떻게 움직이는가	18
검색 처리 순서	18
검색 별칭 사전	18
getSearchText가 합치는 값	18
9. 분류, 그룹, 정렬 이해하기	19
category와 group의 차이	19
그룹을 결정하는 getGroupInfo	19
정렬 옵션	19
10. 즐겨찾기와 브라우저 저장소	20
즐거찾기 저장 흐름	20
localStorage와 sessionStorage	20
11. 로컬 데이터 에디터 사용법	21
에디터의 위치와 연결	21
안전한 곡 수정 순서	21
검사가 확인하는 것	21
12. 직접 해보는 안전한 수정 5가지	22
실습 1: 로고와 검색바 간격 넓히기	22
실습 2: J-POP 색상 바꾸기	22
실습 3: 검색창 안내 문구 바꾸기	22
실습 4: 검색 별칭 하나 추가하기	22
실습 5: 곡에 표시 태그 추가하기	22
13. 오류를 찾는 순서	24

증상별 첫 확인 파일	24
괄호 점검법	24
브라우저 캐시와 ?v=날짜	24
수정 후 최소 점검	24
14. Git과 GitHub Pages	25
로컬과 공개 사이트는 다른 장소입니다	25
Git 용어	25
15. 초보자가 먼저 이해하면 좋은 설계 판단	26
이 구조가 가벼운 이유	26
대신 생기는 한계	26
나중에 파일을 나눈다면	26
16. 연습 문제	27
기초	27
중급	27
정답을 확인하는 기준	27
17. 빠른 참고표	28
문법 기호	28
파일 선택표	28
커밋 전 체크리스트	28
Notepad++에서 델 헤매는 방법	29
마지막으로	30
이 교재의 기준	30

0. 이 책을 사용하는 방법

처음부터 모든 코드를 이해하려 하지 않아도 됩니다. 지금 필요한 만큼 읽고, 아주 작은 변경부터 성공시키는 것이 목표입니다.

세 가지 읽기 경로

목표	먼저 읽을 장	도착점
색, 간격, 글자만 바꾸고 싶다	1 -> 2 -> 4 -> 10	custom.css에서 안전하게 실험
곡 정보를 직접 관리하고 싶다	1 -> 2 -> 5 -> 11	에디터와 data.js 구조 이해
검색과 즐겨찾기까지 이해하고 싶다	1 -> 2 -> 6 -> 7 -> 8	app.js 흐름 추적

가장 중요한 규칙

한 번에 하나만 바꾸고, 브라우저에서 확인하고, 문제가 없을 때 커밋합니다. 여러 곳을 동시에 바꾸면 무엇 때문에 깨졌는지 찾기 어려워집니다.

코드를 읽는 기본 자세

- 파일 이름부터 확인합니다. 같은 단어라도 HTML, CSS, JavaScript에서 역할이 다릅니다.
- 괄호는 짝으로 읽습니다. { }, [], () 중 하나가 빠지면 아래 코드까지 함께 망가질 수 있습니다.
- 영어 단어를 모두 번역하려 하지 말고, 반복해서 등장하는 이름의 역할을 익힙니다.
- 수정 전 검색, 수정 후 저장, 브라우저 확인을 습관으로 만듭니다.

1. 웹사이트를 보는 가장 쉬운 관점

브라우저는 HTML을 읽어 뼈대를 만들고, CSS로 꾸민 뒤, JavaScript를 실행해 움직임을 붙입니다. Flylist는 여기에 data.js라는 곡 창고를 따로 둡니다.

Flylist는 정적 사이트입니다

정적이라는 말은 화면이 움직이지 않는다는 뜻이 아닙니다. 서버가 사용자마다 새 HTML을 만들어 보내지 않는다는 뜻에 가깝습니다. Flylist의 검색, 탭, 즐겨찾기는 모두 사용자의 브라우저 안에서 실행됩니다.



별도 데이터베이스 서버 없이도 브라우저 기능으로 충분히 동작합니다.

네 가지 핵심 파일

파일	비유	실제 역할	현재 크기
index.html	건물의 골조	검색창, 탭 자리, 곡 목록 자리	117줄
styles.css	인테리어 설계	색상, 간격, 카드, 모바일 배치	1,594줄
data.js	곡 창고	TJ 번호, 제목, 아티스트, 태그	11,814줄
app.js	운영 담당자	검색, 분류, 정렬, 즐겨찾기, 렌더링	1,281줄

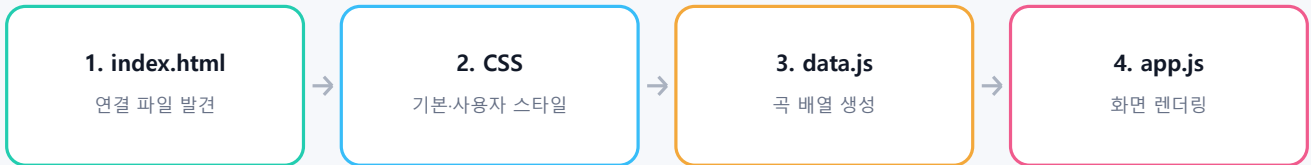
핵심 문장

HTML은 무엇이 있는지, CSS는 어떻게 보이는지, JavaScript는 무엇을 하는지, data.js는 어떤 곡이 있는지를 담당합니다.

2. 현재 Flylist의 전체 구조

파일을 따로 보지 말고, 브라우저가 어떤 순서로 읽는지 연결해서 이해해 봅니다.

브라우저가 페이지를 여는 순서



data.js가 먼저 실행되어야 app.js가 곡 목록을 사용할 수 있습니다.

index.html 8-11줄

```

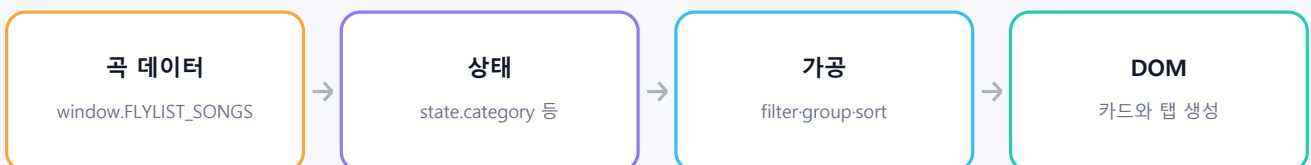
1 <link rel="stylesheet" href="/styles.css?v=20260730-2">
2 <link rel="stylesheet" href="/custom.css?v=20260722-2">
3 <script src="/data.js?v=20260730-2" defer></script>
4 <script src="/app.js?v=20260730-3" defer></script>
  
```

위에서 아래 순서가 중요합니다. custom.css는 styles.css 뒤에 와서 기존 디자인을 덮어쓸 수 있습니다.

나머지 파일의 역할

- custom.css: 실험과 개인 조정을 위한 얇은 덮어쓰기 파일입니다. 현재 64줄이라 styles.css보다 읽기 쉽습니다.
- .nojekyll: GitHub Pages가 파일을 가공하지 않고 그대로 전달하도록 돕는 빈 표식 파일입니다.
- README.md: 프로젝트 목적과 주요 파일을 설명합니다. 사이트 동작에는 직접 참여하지 않습니다.
- .gitignore: 공개 저장소에 올리지 않을 로컬 에디터 파일 이름을 기록합니다.
- 로컬 에디터: C:/Users/ctr12/Documents/PC-Tools/flylist-editor에 따로 있으며 GitHub Pages에는 올라가지 않습니다.

화면을 만드는 핵심 흐름



원본 데이터를 바로 수정하는 대신, 상태에 맞게 골라 화면 요소를 새로 만듭니다.

3. HTML 문법: 화면의 뼈대 읽기

HTML은 프로그래밍 언어라기보다 의미와 구조를 표시하는 마크업 언어입니다. 태그, 속성, 부모-자식 관계를 이해하면 충분히 읽을 수 있습니다.

태그의 기본 모양

HTML 한 줄 해부

```
1 <section id="songList" class="song-list" aria-live="polite"></section>
```

section은 태그, id와 class는 속성, songList와 song-list는 속성값입니다.

부분	뜻	Flylist에서의 사용
	하나의 의미 있는 영역	곡 목록, 업데이트 요약, 즐겨찾기
id="songList"	페이지에서 하나뿐인 이름	JavaScript가 정확히 찾아가는 표지판
class="song-list"	여러 요소가 공유할 수 있는 분류	CSS가 같은 디자인을 적용하는 연결점
aria-live	보조 기술용 설명	검색 결과 변화를 화면 읽기 도구에 전달

부모와 자식

간단히 줄인 index.html

```
1 <header class="app-header">
2   <form class="search">
3     <input id="searchInput" type="search">
4   </form>
5   <nav id="categoryTabs" class="tabs"></nav>
6 </header>
```

들여쓰기가 깊어질수록 안쪽 자식입니다. form과 nav는 header 안에 있습니다.

Flylist에서 자주 보는 HTML 태그

태그	쉬운 뜻	현재 사용처
header	상단 영역	로고, 검색, 카테고리
main	페이지의 핵심 내용	곡 목록과 도구 모음
nav	이동·선택 메뉴	카테고리 탭, 빠른 이동
form	입력 묶음	검색창
button	명령 버튼	즐거찾기, 뒤로가기, 목차
input	사용자 입력	검색어, 파일 선택
section	내용 구역	목록, 즐거찾기 화면
footer	하단 정보	검증 기준일

hidden과 defer

hidden은 요소를 삭제하지 않고 잠시 감춥니다. 즐거찾기 화면과 빠른 이동 창이 이 방식으로 나타났다 사라집니다. defer는 HTML 구조를 먼저 읽은 뒤 JavaScript를 실행하라는 뜻입니다.

실수 방지

태그를 열었다면 닫는 태그도 필요합니다. 을 열고 을 빠뜨리면 뒤의 영역이 전부 그 안으로 들어간 것처럼 해석될 수 있습니다.

4. CSS 문법: 색과 배치 바꾸기

CSS는 선택자로 대상을 고른 뒤 속성과 값을 적습니다. Flylist의 시각 디자인 대부분은 styles.css에 있고, 개인 실험은 custom.css에서 하는 편이 안전합니다.

선택자, 속성, 값

CSS 기본 문법

```
1 .search {
2   margin-top: 24px;
3 }
4
5 .song-card {
6   border-radius: 8px;
7   background: #151923;
8 }
```

.search와 .song-card는 선택자입니다. 중괄호 안에는 속성: 값; 형태로 적습니다.

기호	의미	예
.	class 선택	.song-card
#	id 선택	#searchInput
:	상태나 특수 조건	:hover, :focus
[]	속성 기준 선택	[data-accent="J-POP"]
공백	안쪽 자식 선택	.search input

색상 변수는 조종석입니다

styles.css 1-13줄, 615줄 부근

```
1 :root {
2   --miku: #39f5d1;
3   --virtual: #a78bfa;
4   --anime: #ffb454;
5   --jpop: #ff5c8a;
6 }
7
8 [data-accent="J-POP"] {
9   --accent: var(--jpop);
10 }
```

같은 색을 여러 곳에 직접 쓰지 않고 변수 이름으로 재사용합니다.

박스 모델

화면 요소는 내용(content)을 중심으로 안쪽 여백(padding), 테두리(border), 바깥 여백(margin)을 가집니다. 검색바와 로고 사이를 넓히고 싶다면 검색바의 margin-top을 조정하는 이유가 여기에 있습니다.

속성	영향	초보자가 자주 쓰는 값
margin	요소 바깥 간격	margin-top: 24px
padding	내용과 테두리 사이	padding: 10px 14px
gap	flex/grid 자식 사이	gap: 8px
width / max-width	가로 크기 제한	width: 100%
min-height	최소 높이	min-height: 42px
border-radius	모서리 둥글기	border-radius: 8px

모바일 규칙: 미디어 쿼리

화면 조건에 따른 CSS

```

1 @media (max-width: 719px) {
2   .app-header {
3     position: static;
4   }
5 }
6
7 @media (prefers-reduced-motion: reduce) {
8   * { animation-duration: 0.01ms !important; }
9 }
```

719px 이하에서는 모바일 규칙이 적용되고, 모션 줄이기 설정도 존중합니다.

custom.css를 먼저 쓰는 이유

styles.css는 전체 설계도라 길고 영향 범위가 큼니다. custom.css는 뒤에서 읽히므로 같은 선택자를 다시 적어 작은 실험을 덮어쓸 수 있습니다.

5. data.js 문법: 곡 데이터 읽기

data.js는 거대한 배열 안에 곡 객체가 차례대로 들어 있는 구조입니다. 한 곡은 하나의 객체이며, 속성 이름과 값의 쌍으로 정보를 보관합니다.

배열과 객체

data.js 실제 구조를 한 곡만 남긴 예

```
1 window.FLYLIST_SONGS = [
2   {
3     "number": "28424",
4     "titleKo": "연애재판",
5     "titleOriginal": " 愛裁判",
6     "artist": "40mP feat. 初音ミク",
7     "tag": "40mP",
8     "category": "보카로",
9     "group": "",
10    "tags": ["하츠네 미쿠"]
11  }
12 ];
```

대괄호 []는 곡 전체 목록, 중괄호 { }는 한 곡, 따옴표 안은 문자열입니다.

곡 필드 사전

필드	필수	뜻	예
number	예	TJ 번호	28424
titleKo	예	사이트에서 크게 보일 제목	연애재판
titleOriginal	예	원제	愛裁判
artist	예	TJ 기준 아티스트 표기	40mP feat. 初音ミク
tag	예	기본 묶음·검색 표기	40mP
tagKo	아니오	한국어 별칭	데코니나
category	예	주 분류	보카로
group	아니오	자동 묶음 대신 쓸 소분류	스텔라이브
tags	아니오	추가 검색·표시 태그 배열	["하츠네 미쿠"]
alsoCategories	아니오	추가로 나타날 분류	["애니메이션"]
updateType	아니오	신규 또는 수정	new / updated
updatedAt	아니오	업데이트 날짜	2026-07-30
updateNote	아니오	변경 이유	TJ 공식 정보 교정

쉼표가 중요한 이유

객체 안에서 다음 속성이 이어지면 이전 줄 끝에 쉼표가 필요합니다. 마지막 속성 뒤의 쉼표는 이 프로젝트의 JSON 스타일에서는 보통 생략합니다. 곡 객체와 다음 곡 객체 사이에도 쉼표가 필요합니다.

쉼표 위치 예시

```
1 {
2   "number": "12345",    // 다음 속성이 있으므로 쉼표
3   "titleKo": "예시 곡", // 다음 속성이 있으므로 쉼표
4   "category": "J-POP"   // 마지막 속성
5 },                    // 다음 곡이 이어짐
6 +{
7   "number": "12346"
8 +}
```

권장 작업

data.js를 직접 대량 편집하기보다 로컬 데이터 에디터에서 수정하고, 데이터 검사를 통과한 파일을 저장하는 편이 안전합니다.

현재 데이터 한눈에 보기

주 분류	수록 수
보카로	255곡
버추얼 아티스트	103곡
애니메이션	304곡
J-POP	241곡
전체	903곡

if / else와 삼항 연산자

```

1  if (state.view === "favorites") {
2    renderFavorites();
3  } else {
4    renderMain();
5  }
6
7  const label = expanded ? "접기" : "펼치기";

```

조건이 참이면 앞쪽 작업, 거짓이면 뒤쪽 작업을 선택합니다.

map, filter, forEach

메서드	질문	Flylist식 예
map	각 항목을 무엇으로 바꿀까?	곡 -> TJ 번호와 곡을 잇는 Map 항목
filter	어떤 항목만 남길까?	현재 카테고리나 검색어에 맞는 곡
forEach	각 항목마다 무엇을 할까?	버튼마다 클릭 이벤트 연결
sort	어떤 순서로 놓을까?	수록곡 많은 순, 이름 순

Map과 Set

Map은 번호표가 붙은 서랍처럼 키와 값을 연결합니다. Set은 같은 값을 중복해서 넣지 않는 목록입니다. TJ 번호로 곡을 빠르게 찾을 때 Map, 즐겨찾기 번호를 중복 없이 보관할 때 Set이 알맞습니다.

app.js 3줄과 즐겨찾기 처리

```

1  const songsByNumber = new Map(
2    songs.map(song => [song.number, song])
3  );
4
5  state.favorites.add(number);
6  state.favorites.delete(number);

```

7. app.js의 큰 흐름

app.js 1,281줄을 위에서 아래로 전부 외울 필요는 없습니다. 상태, 요소, 초기화, 이벤트, 렌더링이라는 다섯 덩어리로 나누면 읽을 수 있습니다.

1단계: 데이터와 상태 준비

app.js 시작 부분을 단순화한 예

```
1 const songs = Array.isArray(window.FLYLIST_SONGS)
2   ? window.FLYLIST_SONGS
3   : [];
4
5 const state = {
6   query: "",
7   category: "전체",
8   view: "main",
9   favorites: loadSet("flylist:favorites")
10  };
```

songs는 원본 곡 목록, state는 사용자가 현재 보고 선택한 상태입니다.

2단계: HTML 요소에 짧은 이름 붙이기

app.js 32줄 부근

```
1 const els = {
2   searchInput: document.querySelector("#searchInput"),
3   categoryTabs: document.querySelector("#categoryTabs"),
4   songList: document.querySelector("#songList")
5 };
```

매번 document.querySelector를 쓰지 않고 els.searchInput처럼 짧게 사용합니다.

3단계: init으로 시작

app.js 225-236줄을 단순화

```
1 init();
2
3 function init() {
4   pruneFavoriteState();
5   renderStats();
6   renderTabs();
7   bindEvents();
8   syncViewFromLocation();
9 }
```

페이지가 열리면 통계와 탭을 그리고, 클릭과 입력을 들을 준비를 합니다.

4단계: 이벤트를 듣기

app.js 240줄 부근

```
1 els.searchInput.addEventListener("input", () => {  
2   state.query = normalize(els.searchInput.value);  
3   window.clearTimeout(searchTimer);  
4   searchTimer = window.setTimeout(renderMain, 100);  
5 });
```

검색창 내용이 바뀌면 상태를 갱신하고 0.1초 뒤 목록을 다시 그립니다.

5단계: render 함수로 화면 만들기



상태가 바뀌면 원본 HTML을 손으로 고치는 대신 JavaScript가 필요한 카드를 다시 만듭니다.

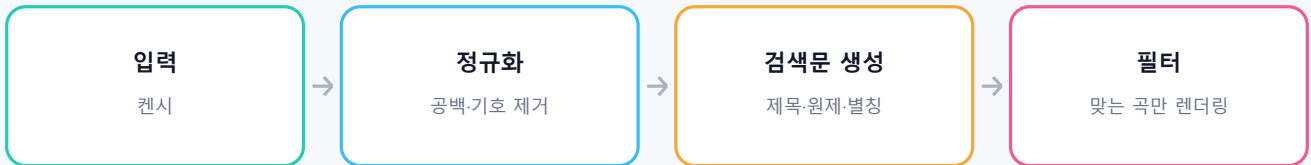
이 구조의 장점

index.html에는 903개의 곡 카드가 직접 적혀 있지 않습니다. 빈 songList 자리에 app.js가 data.js를 읽어 필요한 카드만 만들어 넣습니다.

8. 검색은 실제로 어떻게 움직이는가

검색은 단순히 글자가 같은지 보는 것이 아니라, 여러 필드를 하나의 검색용 문자열로 합친 뒤 표기를 정규화해 비교합니다.

검색 처리 순서



검색 별칭 덕분에 米津玄師를 켄시로도 찾을 수 있습니다.

검색 별칭 사전

app.js 191줄 부분

```

1  const searchAliasMap = {
2    "DECO*27": ["데코27", "데코 니나"],
3    "初音ミク": ["미쿠"],
4    "米津玄師": ["켄시"],
5    "ASIAN KUNG-FU GENERATION": ["아지강"]
6  };

```

카드에 모두 표시하지 않고 검색할 때만 사용하는 별칭입니다.

getSearchText가 합치는 값

TJ 번호, 한국어 제목, 원제, 아티스트, 기본 태그, 한국어 태그, 추가 태그, 별칭, 분류, 그룹을 모두 합칩니다. 따라서 사용자는 한 가지 정확한 표기만 외울 필요가 없습니다.

app.js 1054줄 부분을 단순화

```

1  const value = normalize([
2    song.number,
3    song.titleKo,
4    song.titleOriginal,
5    song.artist,
6    ...getSearchAliasCandidates(song),
7    song.category,
8    song.group
9  ].join(" ");

```

별칭을 추가할 때

원문 이름을 왼쪽 키로 두고, 사용자가 실제로 입력할 가능성이 높은 표현만 오른쪽 배열에 넣습니다. 너무 짧은 별칭은 관계없는 곡까지 검색될 수 있습니다.

9. 분류, 그룹, 정렬 이해하기

Flylist는 곡 하나를 보여주는 것뿐 아니라, 현재 탭에 맞는 그룹을 만들고 수록곡 수나 이름으로 정렬합니다.

category와 group의 차이

값	범위	예
category	큰 탭	보카로, 버추얼 아티스트, 애니메이션, J-POP
group	탭 안의 소분류	스텔라이브, Fate 시리즈, 작품명
tag	기본 제작자·검색 표기	DECO*27, 40mP
alsoCategories	추가 노출 탭	주 분류 외에 함께 보여야 할 경우

그룹을 결정하는 getGroupInfo

보카로에서는 프로듀서, 애니메이션에서는 작품 그룹, J-POP에서는 아티스트가 기본 묶음이 됩니다. data.js의 group 값이 있으면 자동 기준보다 수동 그룹을 우선할 수 있습니다.

정렬 옵션

- count-desc: 그룹의 수록곡이 많은 순서
- count-asc: 그룹의 수록곡이 적은 순서
- group-name: 소분류의 가수·작품명 순서

app.js 563줄 부근의 정렬 핵심

```

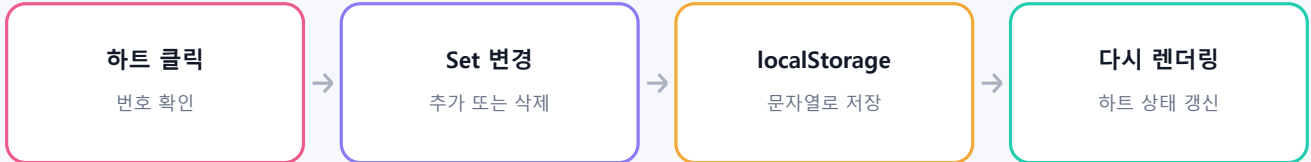
1  if (state.sort === "count-desc"
2    && a.items.length !== b.items.length) {
3    return b.items.length - a.items.length;
4  }
5  return collator.compare(left, right);

```

10. 즐겨찾기와 브라우저 저장소

서버가 없어도 즐겨찾기가 유지되는 이유는 브라우저가 제공하는 localStorage에 번호 목록을 저장하기 때문입니다.

즐거찾기 저장 흐름



브라우저를 닫았다 열어도 같은 기기·같은 브라우저라면 남습니다.

저장 원리를 단순화한 예

```

1 function saveSet(key, values) {
2   localStorage.setItem(key, JSON.stringify([...values]));
3 }
4
5 function loadSet(key) {
6   return new Set(JSON.parse(localStorage.getItem(key) || "[]"));
7 }
  
```

localStorage와 sessionStorage

저장소	현재 용도	언제 사라질 수 있나
localStorage	꼭·아티스트 즐겨찾기	사이트 데이터 삭제, 브라우저 변경, 다른 기기
sessionStorage	탭, 정렬, 펼친 그룹, 스크롤 위치	브라우저 탭 세션 종료

중요한 한계

즐거찾기는 GitHub 저장소나 운영자의 서버 용량을 쓰지 않습니다. 사용자의 브라우저에만 있으므로 기기를 바꾸면 자동으로 따라가지 않습니다. 그래서 백업·복원 기능이 있습니다.

11. 로컬 데이터 에디터 사용법

곡 데이터는 긴 data.js를 직접 찾기보다 로컬 에디터에서 검색·수정·검사하는 편이 안전합니다. 에디터는 공개 저장소 밖에 있습니다.

에디터의 위치와 연결

위치: C:/Users/ctr12/Documents/PC-Tools/flylist-editor

에디터의 editor.html은 YF-1/data.js를 읽습니다. 수정 후 저장한 data.js를 YF-1 폴더의 기존 data.js와 교체하면 index.html이 다음 실행부터 새 데이터를 사용합니다.

안전한 곡 수정 순서

- 1. 에디터에서 곡 번호나 제목을 검색합니다.
- 2. 한국어 제목, 원제, 아티스트, 분류, 그룹, 태그를 수정합니다.
- 3. 변경 적용을 누릅니다.
- 4. 데이터 검사를 실행합니다.
- 5. 오류가 0인지 확인하고, 확인 항목은 의도한 예외인지 살핍니다.
- 6. data.js를 저장하고 Flylist 로컬 화면에서 검색합니다.

검사가 확인하는 것

검사	뜻	자동 수정 여부
TJ 번호 중복	서로 다른 곡이 같은 번호를 쓰는지	수정하지 않고 오류 표시
필수값 누락	제목·아티스트·분류 등이 비었는지	오류 표시
한국어 표기	일본어만 있고 검색용 한국어가 없는지	확인 항목 표시
그룹·아티스트 통일	같은 대상으로 보이는 표기가 갈리는지	확인 항목 표시
동일 원제	커버곡 또는 동명곡 가능성	확인 항목 표시

왜 자동 수정하지 않나

표기가 다르다고 항상 오류는 아닙니다. 커버곡, 버전 차이, 작품별 그룹처럼 사람의 판단이 필요한 경우가 있어 검사는 후보만 보여줍니다.

12. 직접 해보는 안전한 수정 5가지

아래 실습은 영향 범위가 작은 순서입니다. 한 실습이 끝날 때마다 저장하고 브라우저에서 확인하세요.

실습 1: 로고와 검색바 간격 넓히기

가장 안전한 CSS 실험

```
1 /* custom.css */
2 .search {
3   margin-top: 28px;
4 }
```

현재 값보다 크게 하면 간격이 넓어집니다. 마음에 들지 않으면 이 세 줄만 지우면 됩니다.

실습 2: J-POP 색상 바꾸기

CSS 변수 덮어쓰기

```
1 /* custom.css */
2 :root {
3   --jpop: #ff6fa0;
4 }
```

styles.css를 직접 고치지 않고 custom.css에서 색 변수를 덮어씁니다.

실습 3: 검색창 안내 문구 바꾸기

index.html 29줄 부근

```
1 <input id="searchInput"
2   type="search"
3   placeholder="번호, 제목, 원제, 아티스트 검색">
```

placeholder 값만 바꿉니다. 따옴표와 꺾쇠는 그대로 둡니다.

실습 4: 검색 별칭 하나 추가하기

app.js 191줄 부근

```
1 const searchAliasMap = {
2   ...기존 항목들,
3   "새 원문 이름": ["한국어 별칭", "짧은 별칭"]
4 };
```

바로 윗줄 끝의 쉼표가 있는지 확인합니다. 너무 일반적인 한 글자 별칭은 피합니다.

실습 5: 곡에 표시 태그 추가하기

data.js의 한 곡 객체

```
1 "group": "",  
2 +"tags": [  
3   "하츠네 미쿠",  
4   "커버곡"  
5 +]
```

에디터의 추가 표시 태그 기능을 쓰는 편이 더 안전합니다.

13. 오류를 찾는 순서

코드가 깨졌을 때 무작정 여러 곳을 고치면 더 어려워집니다. 증상에서 출발해 가장 가까운 파일부터 확인합니다.

증상별 첫 확인 파일

증상	먼저 볼 파일	대표 원인
글자나 색만 이상함	custom.css -> styles.css	선택자, 속성값, 닫는 중괄호
꼭 하나의 정보가 이상함	data.js 또는 에디터	필드 오타, 잘못된 번호·분류
검색 결과가 이상함	app.js의 별칭·getSearchText	별칭 누락, 너무 짧은 별칭
버튼이 아무 반응 없음	app.js bindEvents	id 변경, JavaScript 문법 오류
화면 전체가 비어 있음	data.js -> app.js -> 브라우저 콘솔	괄호·심표 오류, 파일 경로
로컬은 되는데 공개 사이트는 예전	Git 상태와 GitHub Pages	커밋·푸시 누락, 캐시
모바일에서만 깨짐	@media 규칙	고정 너비, 긴 글자, overflow

괄호 점검법

에러가 난 줄이 진짜 원인이라는 보장은 없습니다. 위쪽에서 따옴표나 괄호가 닫히지 않으면 브라우저는 한참 아래에서야 해석을 포기할 수 있습니다. 수정한 지점의 위아래 10줄을 함께 봅니다.

브라우저 캐시와 ?v=날짜

index.html의 파일 주소 뒤 ?v=20260730-3 같은 값은 캐시 구분표입니다. CSS나 JavaScript를 배포했는데 예전 파일이 계속 보이면 이 값을 새 날짜나 버전으로 바꾸면 브라우저가 새 파일로 인식하기 쉽습니다.

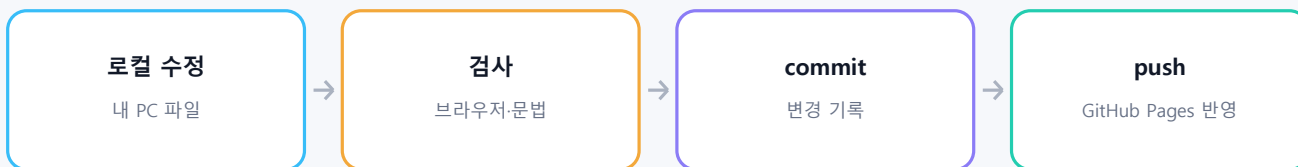
수정 후 최소 점검

- 전체 탭이 열리는가
- 검색어 한글·일본어·TJ 번호가 각각 동작하는가
- 카테고리 탭과 즐겨찾기가 동작하는가
- 모바일 폭에서 가로 스크롤이 생기지 않는가
- 새로고침 후 즐겨찾기와 현재 상태가 의도대로 유지되는가
- 브라우저 개발자 도구 Console에 빨간 오류가 없는가

14. Git과 GitHub Pages

로컬 파일을 수정했다고 공개 사이트가 즉시 바뀌지는 않습니다. 변경을 기록하는 commit과 GitHub로 보내는 push가 필요합니다.

로컬과 공개 사이트는 다른 장소입니다



push 이후에도 GitHub Pages 배포와 브라우저 캐시에 잠시 시간이 필요할 수 있습니다.

Git 용어

용어	쉬운 뜻	주의
repository	프로젝트와 변경 기록 보관함	Flylist 저장소와 다른 폴더를 혼동하지 않기
status	현재 바뀐 파일 목록	커밋 전에 반드시 확인
stage	이번 커밋에 넣을 파일 선택	관련 파일만 명시적으로 선택
commit	변경 묶음에 이름과 시점 기록	아직 GitHub로 전송된 것은 아님
push	로컬 커밋을 GitHub에 전송	완료 후 원격 커밋 일치 확인

개념을 보여주는 기본 흐름

- 1 `git status --short`
- 2 `git add app.js data.js index.html styles.css`
- 3 `git commit -m "feat: describe the change"`
- 4 `git push origin main`

실제 작업에서는 바뀐 파일을 확인한 뒤 필요한 파일만 stage합니다.

검증 기준일

Flylist 하단의 DATA VERIFIED 날짜는 공개 커밋 날짜에 맞춰 갱신합니다. 데이터의 최신성을 약속하는 날짜라기보다, 현재 공개본을 마지막으로 검토하고 기록한 기준일입니다.

15. 초보자가 먼저 이해하면 좋은 설계 판단

현재 구조는 최신 프레임워크를 쓰지 않지만, 개인용 정적 아카이브라는 목적에는 분명한 장점이 있습니다.

이 구조가 가벼운 이유

- 외부 프레임워크와 패키지를 내려받지 않습니다.
- 서버 데이터베이스와 로그인 시스템이 없습니다.
- 곡 정보는 data.js 한 파일에서 바로 읽습니다.
- 검색과 즐겨찾기는 브라우저 자체 기능으로 처리합니다.
- GitHub Pages가 정적 파일을 그대로 전달합니다.

대신 생기는 한계

- data.js가 커질수록 처음 내려받는 용량과 직접 검토 부담이 늘어납니다.
- 즐겨찾기는 기기 간 자동 동기화되지 않습니다.
- 모든 코드가 전역 파일에 모이면 app.js가 길어져 탐색이 어려워집니다.
- 자동 업데이트를 하려면 별도의 수집·검증·배포 절차가 필요합니다.

나중에 파일을 나눈다면

당장 분리할 필요는 없지만, 직접 코딩하는 비중이 늘면 app.js를 search.js, favorites.js, render.js처럼 역할별로 나누는 방향을 생각할 수 있습니다. 다만 파일을 나누면 로딩 순서와 공유 값 관리도 함께 설계해야 하므로, 먼저 현재 흐름을 이해한 뒤 진행하는 것이 좋습니다.

좋은 개선의 기준

새 기술을 쓰는 것이 곧 개선은 아닙니다. 사용자가 얻는 편의, 유지보수 난이도, 모바일 안정성, 데이터 신뢰도를 함께 보고 선택합니다.

16. 연습 문제

정답을 바로 보지 말고, 해당 파일에서 단어를 검색해 위치를 찾아보세요. 실제 수정은 복사본이나 custom.css에서 시작하는 편이 안전합니다.

기초

- index.html에서 검색창의 안내 문구를 찾아 다른 문장으로 바꿔 보세요.
- custom.css에서 --anime 색상만 바꾸고 애니메이션 탭과 카드의 변화를 관찰하세요.
- data.js에서 TJ 28424의 titleKo, titleOriginal, artist, tag를 찾아 각각의 역할을 말해 보세요.
- app.js에서 categories 배열을 찾고, 이 배열이 탭과 어떤 관계가 있는지 설명해 보세요.

중급

- searchAliasMap에 테스트 별칭을 하나 추가하고 검색한 뒤 다시 제거하세요.
- renderMain에서 getMainSongs가 호출되는 위치를 찾고, 검색 전후 곡 수가 달라지는 이유를 설명하세요.
- 즐겨찾기 버튼을 누를 때 시작하는 함수부터 localStorage 저장 함수까지 호출 흐름을 적어 보세요.
- @media (max-width: 719px) 영역에서 모바일만 바뀌는 규칙 하나를 찾아 데스크톱과 비교하세요.

정답을 확인하는 기준

코드 한 줄을 똑같이 외웠는지가 아니라, 어느 파일의 어느 역할을 바꿨고 브라우저가 어떤 순서로 반영했는지를 말할 수 있으면 성공입니다.

질문	확인할 핵심
문구 변경	HTML 속성값만 바뀌었는가
색상 변경	custom.css가 styles.css 뒤에서 덮어썼는가
곡 필드	한 객체 안의 키-값 구조를 구분했는가
검색 별칭	원문 키와 별칭 배열, 심표를 지켰는가
즐거찾기	이벤트 -> state -> 저장 -> 렌더링 순서를 찾았는가

17. 빠른 참고표

작업 중 막혔을 때 이 장부터 펼쳐도 됩니다.

문법 기호

기호	이름	역할
{ }	중괄호	CSS 규칙, JavaScript 객체·함수 범위
[]	대괄호	배열, 목록
()	소괄호	함수 호출, 조건, 전달값
" "	따옴표	문자열
:	콜론	객체의 키와 값, CSS 속성과 값 연결
;	세미콜론	CSS 선언과 JavaScript 문장의 끝
,	쉼표	배열 항목과 객체 속성 구분
.	점	객체 속성이나 메서드 접근
=>	화살표	짧은 함수 표현
...	스프레드	배열·객체 내용을 펼치기
?.	옵셔널 체이닝	값이 없을 수 있을 때 안전하게 접근
===	엄격 비교	값과 자료형이 같은지 확인

파일 선택표

하고 싶은 일	수정 파일
문구, 태그 자리, 화면 영역	index.html
색상, 간격, 크기, 모바일 배치	먼저 custom.css, 필요하면 styles.css
곡 번호, 제목, 아티스트, 그룹, 태그	로컬 에디터 -> data.js
검색 별칭, 탭 행동, 즐겨찾기, 정렬	app.js
프로젝트 설명	README.md
공개 반영	Git commit과 push

커밋 전 체크리스트

- 변경 파일 목록을 확인했다.
- 의도하지 않은 파일과 로컬 전용 에디터가 포함되지 않았다.
- JavaScript 문법 검사를 통과했다.

- data.js의 TJ 번호 중복과 필수값 누락이 없다.
- 데스크톱과 모바일에서 주요 흐름을 확인했다.
- 검증 기준일과 캐시 버전을 필요한 경우 갱신했다.
- 커밋 뒤 원격 main과 로컬 커밋이 일치한다.

Notepad++에서 덜 헤매는 방법

작업	방법	이유
원하는 곳 찾기	Ctrl+F로 id, class, 함수 이름 검색	스크롤로 직접 찾는 것보다 빠르고 정확함
같은 단어 모두 보기	찾기 결과 창에 모두 찾기	검색어가 여러 파일·여러 위치에 쓰이는지 확인
괄호 짝 확인	괄호 옆에 커서를 두고 대응 괄호 강조 확인	중괄호나 대괄호 누락 발견
긴 줄 읽기	보기 메뉴에서 자동 줄 바꿈을 잠시 사용	가로 스크롤 부담 감소
문자 깨짐 예방	인코딩을 UTF-8로 유지	한국어·일본어 데이터 보존
수정 취소	저장 전 Ctrl+Z, 저장 후에는 Git diff 확인	의도하지 않은 대량 변경 방지

전체 바꾸기 주의

Replace All은 같은 이름을 가진 다른 곡과 코드까지 바꿀 수 있습니다. 먼저 모두 찾기로 범위를 확인하고, 가능하면 한 항목씩 바꿉니다.

마지막으로

지금 Flylist를 직접 만든 경험은 이미 코딩 학습의 중요한 절반입니다. 무엇이 필요하고, 어떤 동작이 불편하며, 어떤 결과가 맞는지를 판단해 왔기 때문입니다. 이제 그 판단을 코드의 작은 단위와 연결하면 됩니다.

다음 목표

custom.css의 작은 변경 -> HTML 문구 변경 -> 데이터 에디터 사용 -> 검색 별칭 추가
-> app.js 함수 하나 추적 순서로 진행하세요. 매번 작게 바꾸고 확인하면 됩니다.

이 교재의 기준

저장소: C:/Users/ctr12/Documents/YF-1

기준 커밋: b1312c4

커밋 날짜: 2026-07-30

곡 수: 903곡

작성일: 2026-08-01

Oort · Flylist